

# 1. I-Combinator in Javascript

---

The **I-Combinator** (or **identity function**) returns its argument unchanged without any modification.

## Lambda Calculus Equivalence:

```
λx.x
```

### Use Cases:

- Default callback function
- Unchanged transformation
- Building block in combinatory logic and functional programming

---

## 2. Alpha Equivalence

---

Alpha equivalence means lambda expressions are equivalent if they differ only in variable names. Renaming bound variables doesn't change the function's meaning.

### 2.1 Which lambda expression is alpha equivalent to $\lambda x.x$ ?

**Answer:**  $\lambda a.a$

For an input x, return x.  $\lambda a.a$  is basically the same, for an input a, return a.

---

### 2.2 Which lambda expression is alpha equivalent to $\lambda xy.yx$ ?

**Answer:**  $\lambda a.(\lambda b.ba)$

This can be reconstructed to  $\lambda x.(\lambda y.(yx))$  which means take two inputs, x and y, and then return yx. This equivalence just changes the letters x -> a and y -> b, and accomplishes the same thing.

---

### 2.3 Which lambda expression is alpha equivalent to $\lambda xy.xz$ ?

**Answer:**  $\lambda mn.mz$

This can be reconstructed to  $\lambda x.(\lambda y.(xz))$ . We can see that x is bound and y is bound, while z is free since no lambda introduces z. This lambda equation is the same as x -> m and y -> n, but the free z must stay as z.

---

## 3. Applying Beta Reduction to Lambda Functions

---

Beta reduction replaces bound variables with their arguments.

### 3.1 $(\lambda x. x)y$

Apply  $\lambda x.x$  to the input  $y$ . Therefore  $(\lambda x.x)y = y$

---

### 3.2 $\lambda x. xx$

We cannot reduce this as  $\lambda x.xx$  has no input after it, it is only a function definition. So,  $\lambda x.xx$  is already in normal form.

If we were to give it an input and apply it to itself, we have  $(\lambda x.xx)(\lambda x.xx)$ . If we replace both occurrences of  $x$  in  $xx$  with the input  $(\lambda x.xx)$ , we get  $(\lambda x.xx)(\lambda x.xx)$ , which is the exact same expression. If we reduce it again, the same output will be produced, so it never reaches an answer, meaning it diverges.

---

### 3.3 $(\lambda z. zz)(\lambda y. yy)$

Replace both occurrences of  $z$  in  $zz$  with the input,  $(\lambda y.yy)$ . We get  $(\lambda y.yy)(\lambda y.yy)$ . Now, we have the same kind of function applied to itself, if we replace both occurrences of  $y$  in  $yy$  with the input  $(\lambda y.yy)$ , we get  $(\lambda y.yy)(\lambda y.yy)$ . Once again, the expression produces itself, meaning this repeats forever, so it never reaches a normal form and it diverges.

---

### 3.4 $(\lambda x. xx)y$

Replace both occurrences of  $x$  in  $xx$  with the input,  $y$ . We get  $yy$ . There is no more reduction since  $y$  is just a variable, not a lambda function. So, the normal form is  $yy$ .

---

## 4. Beta Reduce the Following Expressions

---

### 4.1 $(\lambda y. zy)a$

Replace  $y$  with the input  $a$ , we get  $za$ . No more lambda function, so  $za$  is the normal form.

---

### 4.2 $(\lambda x. x)(\lambda x. x)$

Replace the  $x$  in the first function with the input,  $(\lambda x.x)$ . We get  $\lambda x.x$ . No input is given, so  $\lambda x.x$  is the normal form.

---

### 4.3 $(\lambda x. xy)(\lambda x. xx)$

Replace the first  $x$  with the input,  $(\lambda x.xx)$ . We get  $(\lambda x.xx)y$ . We can apply the function again, replacing both  $x$  in  $xx$  with  $y$ . We get  $yy$ . The normal form is  $yy$  since we no longer have any lambda functions.

---

### 4.4 $(\lambda z. z)(\lambda a. aa)(\lambda z. zb)$

Replace  $z$  with the first input,  $(\lambda a.aa)$ . We get  $(\lambda a.aa)(\lambda z.zb)$ . Replace both  $a$  in  $aa$  with the second input,  $(\lambda z.zb)$ . We get  $(\lambda z.zb)(\lambda z.zb)$ . Replace  $z$  in  $zb$  with the third input,  $(\lambda z.zb)$ . We get  $(\lambda z.zb)b$ . Replace  $z$  in  $zb$  with  $b$ , we get  $bb$ . No more lambda functions are present,  $bb$  is the normal form.

---

## 5. Eta Conversion

---

Given a lambda function  $\lambda x.(f\ x)$ , we can simply take the function  $f$  by doing an eta reduction.

### 5.1 $\lambda x.zx$

$\lambda x.zx$  follows the pattern of  $\lambda x.(f\ x)$ , so we can reduce  $\lambda x.zx$  to  $z$ .

---

### 5.2 $\lambda x.xz$

We cannot eta reduce this function as  $x$  is being used as the function.

---

### 5.3 $(\lambda x.bx)(\lambda y.ay)$

Eta reduce  $(\lambda x.bx)$  to get  $b$ . Eta reduce  $(\lambda y.ay)$  to get  $a$ . So we get  $ba$  after it is reduced.

---

## 6. Which of the Following are Combinators?

---

### 6.1 $\lambda x.xxx$

This function is a combinator as all  $x$ 's are bounded by  $\lambda x$ .

---

### 6.2 $\lambda xy.zx$

This function is not a combinator as  $x$  and  $y$  are bounded but  $z$  isn't.

---

### 6.3 $\lambda xyz.xy(zx)$

This function is a combinator because all variables  $x$ ,  $y$ , and  $z$  are all bounded.

---

### 6.4 $\lambda xyz.xy(zxy)$

This function is a combinator because all variables  $x$ ,  $y$ , and  $z$  are all bounded.

---

## 7. Apply $g$ to Y-Combinator

---

$$Y = \lambda f. (\lambda x. f(xx)) (\lambda x. f(xx))$$

Apply  $g$  to it, so  $\lambda f. (\lambda x. f(xx)) (\lambda x. f(xx)) g$ . Replace  $f$  with  $g$ ,  $(\lambda x. g(xx)) (\lambda x. g(xx))$ . Replace  $x$  with  $(\lambda x. g(xx))$ ,  $g((\lambda x. g(xx)) (\lambda x. g(xx)))$ .

$(\lambda x. g(xx)) (\lambda x. g(xx))$  is  $Y$  applied with  $g$ , so we can substitute this to  $g(Y g)$ .

If we continue reducing this, we will get  $Y g \rightarrow g(Y g) \rightarrow g(g(Y g)) \rightarrow \dots$

So, we can't simplify it further without actually knowing what  $g$  is.

## 8. Church Encoding

Using:

```
TRUE = \xy. x
FALSE = \xy. y
IF = \btf. b t f
AND = \xy. IF x y FALSE
OR = \xy. IF x TRUE y
NOT = \x. IF x FALSE TRUE
```

### 8.1 Evaluate NOT FALSE

$$\text{NOT} = \lambda x. \text{IF } x \text{ FALSE TRUE}$$

Apply it to FALSE, so  $(\lambda x. \text{IF } x \text{ FALSE TRUE}) \text{ FALSE}$ . Replace  $x$  with FALSE, we get  $\text{IF FALSE FALSE TRUE}$ .

Use  $\text{IF} = \lambda btf. b t f$ ,  $(\lambda btf. b t f) \text{ FALSE FALSE TRUE}$ , we get  $\text{FALSE FALSE TRUE}$ .

Use  $\text{FALSE} = \lambda xy. y$ , apply  $\text{FALSE TRUE}$ ,  $(\lambda xy. y) \text{ FALSE TRUE}$ , we get  $\text{TRUE}$ .

So,  $\text{NOT FALSE} = \text{TRUE}$

### 8.2 Evaluate OR TRUE FALSE

$$\text{OR} = \lambda xy. \text{IF } x \text{ TRUE } y$$

$(\lambda xy. \text{IF } x \text{ TRUE } y) \text{ TRUE FALSE}$ . Replace  $x$  with TRUE and  $y$  with FALSE. We get  $\text{IF TRUE TRUE FALSE}$ .

Use  $\text{IF} = \lambda btf. b t f$ ,  $(\lambda btf. b t f) \text{ TRUE TRUE FALSE}$ . We get  $\text{TRUE TRUE FALSE}$ .

Use  $\text{TRUE} = \lambda xy. x$ ,  $(\lambda xy. x) \text{ TRUE FALSE}$ . We get  $\text{TRUE}$ .

Therefore,  $\text{OR TRUE FALSE} = \text{TRUE}$

### 8.3 Evaluate AND TRUE TRUE

Use  $AND = \lambda xy. IF\ x\ y\ FALSE$

$(\lambda x. \lambda y. IF\ x\ y\ FALSE)\ TRUE\ TRUE$ , replace  $x$  with  $TRUE$  and  $y$  with  $TRUE$ . We get  $IF\ TRUE\ TRUE\ FALSE$ .

Use  $IF = \lambda btf. b\ t\ f$ ,  $(\lambda btf. b\ t\ f)\ TRUE\ TRUE\ FALSE$ , replace  $b$  with  $TRUE$ ,  $t$  with  $TRUE$ ,  $f$  with  $FALSE$ . We get  $TRUE\ TRUE\ FALSE$ .

Use  $TRUE = \lambda xy. x$ ,  $(\lambda xy. x)\ TRUE\ FALSE$ , replace  $x$  with  $TRUE$ . We get  $TRUE$ .

---

## 9. Translate the Following Lambda Expressions into JavaScript Arrow Functions

---

### 9.1 $\lambda xy. x$

```
x => y => x
```

---

### 9.2 $\lambda fx. f(f\ x)$

```
f => x => f(f(x))
```

---

### 9.3 $\lambda f. \lambda g. \lambda x. f(g\ x)$

```
f => g => x => f(g(x))
```